

fun.lol Product Documentation

Generated: 2026-05-24

This combined document includes the software architecture diagram document, product design document, and product journey map.

Software Architecture Diagram Document

Generated: 2026-05-24

Source inspection scope: 'index.html', 'script.js', 'styles.css', 'server.js', 'package.json', and 'supabase-schema.sql'.

A. Architecture Overview

fun.lol is implemented as a lightweight single-page web application with a Node.js backend. The frontend is delivered from static files and switches between an authenticated dashboard, a bio/profile editor, public profile pages, games, friends, tribes, and tribe chats. The backend exposes JSON API routes for authentication, profile publishing, media delivery, friends, tribes, tribe chat, and the server-backed Snake score.

Confirmed implementation:

- Frontend files: 'index.html', 'script.js', and 'styles.css'.
- Backend entry point: 'server.js', using Node's built-in 'http' module rather than Express or another framework.
- Static routing: '/' serves 'index.html'; '/u/:handle' also serves 'index.html' so the client can load a public profile by handle.
- API routing: '/api/...' routes are handled directly in 'server.js'.
- Local persistence fallback: 'data/users.json' and 'data/profiles.json'.
- Optional Supabase persistence: enabled when 'SUPABASE_URL' and 'SUPABASE_SERVICE_ROLE_KEY' are present.
- Supabase tables currently defined in 'supabase-schema.sql': 'app_users', 'app_sessions', and 'app_profiles'.
- Supabase Storage bucket used by the server when configured: 'profile-media'.
- Public discovery support: '/robots.txt' and '/sitemap.xml'.

Storage model:

- In local mode, users and sessions are stored in 'data/users.json'; profiles and most social features are stored in 'data/profiles.json'.
- In Supabase mode, users are stored in 'app_users', sessions in 'app_sessions', and profile data in 'app_profiles.data' as JSONB.
- Friends, friend requests, tribes, tribe invites, join requests, tribe messages, profile settings, media references, social links, and most profile customization data are logical entities currently nested inside profile JSON.
- Media files are stored as data URLs in profile JSON when Supabase is not configured. When Supabase is configured, uploaded avatar, background, and music files are moved into Supabase Storage and referenced by path.

API Route Summary

Area: Authentication; Routes: 'POST /api/signup', 'POST /api/login', 'GET /api/me'; Confirmed behavior: Creates users, validates credentials, returns session token and

account profile metadata.

Area: Profile; Routes: 'GET /api/my-profile', 'GET /api/profiles/:handle', 'PUT /api/profiles/:handle'; Confirmed behavior: Loads owned profile, loads public profile, publishes profile changes.

Area: Public media; Routes: 'GET /api/profiles/:handle/avatar', '/background', '/music'; Confirmed behavior: Delivers uploaded media from Supabase Storage path or local data URL fallback.

Area: Friends; Routes: 'POST /api/friend-requests', 'POST /api/friend-requests/:id/accept', 'DELETE /api/friends/:key'; Confirmed behavior: Sends, accepts, and removes friendships.

Area: Tribes; Routes: 'GET /api/tribes', 'POST /api/tribes', 'PATCH /api/tribes/:tribeld', 'DELETE /api/tribes/:tribeld'; Confirmed behavior: Lists, creates, updates, and deletes tribes.

Area: Tribe membership; Routes: 'POST /api/tribes/:tribeld/join', 'POST /api/tribe-invites/:id/:action', 'POST /api/tribe-join-requests/:id/:action', 'DELETE /api/tribes/:tribeld/members/:memberId'; Confirmed behavior: Handles invite, join, owner approval, decline, and member removal flows.

Area: Tribe chat; Routes: 'GET /api/tribes/:tribeld/messages', 'POST /api/tribes/:tribeld/messages'; Confirmed behavior: Loads and appends messages for members only.

Area: Games; Routes: 'GET /api/games/snake-score', 'PUT /api/games/snake-score'; Confirmed behavior: Saves the authenticated user's Snake high score.

Area: SEO; Routes: 'GET /robots.txt', 'GET /sitemap.xml'; Confirmed behavior: Allows crawlers and lists the home page plus public profile URLs.

B. System Architecture Diagram

```
mermaid
graph LR
    Visitor[Public visitor] --> Browser[User browser]
    User[Logged-in user] --> Browser

    Browser --> PublicProfile[Public profile page /u/:handle]
    Browser --> Dashboard[Private dashboard]

    Dashboard --> AuthUI[Auth and session UI]
    Dashboard --> ProfileBuilder[Bio/Profile builder]
    Dashboard --> FriendsUI[Friends UI]
    Dashboard --> NotificationsUI[Notifications UI]
    Dashboard --> TribesUI[Tribes UI]
    Dashboard --> TribeChatUI[Tribe chat UI]
    Dashboard --> GamesUI[Games UI]
    ProfileBuilder --> MediaUploadUI[Media upload controls]

    PublicProfile --> ProfileAPI[Profile API]
    AuthUI --> AuthAPI[Auth API]
    ProfileBuilder --> ProfileAPI
    MediaUploadUI --> MediaAPI[Media API]
    FriendsUI --> FriendsAPI[Friends API]
    NotificationsUI --> FriendsAPI
    NotificationsUI --> TribesAPI[Tribes API]
    TribesUI --> TribesAPI
    TribeChatUI --> TribeChatAPI[Tribe chat API]
    GamesUI --> GamesAPI[Games API]

    AuthAPI --> NodeBackend[Node.js server.js]
    ProfileAPI --> NodeBackend
    MediaAPI --> NodeBackend
    FriendsAPI --> NodeBackend
```

```

TribesAPI --> NodeBackend
TribeChatAPI --> NodeBackend
GamesAPI --> NodeBackend

NodeBackend --> LocalJSON[(Local JSON storage)]
NodeBackend --> SupabaseDB[(Optional Supabase database)]
NodeBackend --> SupabaseStorage[(Optional Supabase Storage profile-media)]
NodeBackend --> SEO[Sitemap and robots.txt]

SupabaseDB --> AppUsers[app_users]
SupabaseDB --> AppSessions[app_sessions]
SupabaseDB --> AppProfiles[app_profiles JSONB data]

```

C. Entity Relationship Diagram

This ERD documents the logical product model. Confirmed physical Supabase tables are 'app_users', 'app_sessions', and 'app_profiles'; the other entities are currently represented inside profile JSON or derived from profile JSON.

```

mermaid
erDiagram
    USER ||--o| PROFILE : owns
    USER ||--o{ ACCOUNT_SETTINGS : has
    PROFILE ||--o{ SOCIAL_LINK : contains
    PROFILE ||--o{ MEDIA_ASSET : references
    USER ||--o{ GAME_SCORE : earns
    USER ||--o{ FRIEND_REQUEST : sends
    USER ||--o{ FRIEND_REQUEST : receives
    USER ||--o{ FRIENDSHIP : participates
    USER ||--o{ NOTIFICATION : receives
    USER ||--o{ TRIBE : owns
    TRIBE ||--o{ TRIBE_MEMBER : includes
    TRIBE ||--o{ TRIBE_INVITE : sends
    TRIBE ||--o{ TRIBE_JOIN_REQUEST : receives
    TRIBE ||--o{ TRIBE_MESSAGE : contains
    USER ||--o{ TRIBE_MESSAGE : sends

    USER {{
        string id
        string email
        string password_hash
        string profile_handle
        string profile_path
        string profile_url
        integer snake_high_score
        datetime created_at
    }}

    PROFILE {{
        string handle
        string owner_user_id
        integer views
        string name
        string bio
    }}

```

```
    string location
    string theme
    boolean compactLinks
    boolean animatedBackground
    boolean darkVideo
    boolean cursorTrail
    string cursorColor
    string sparkleEffect
    datetime updatedAt
}}
```

```
SOCIAL_LINK {{
    string platform
    string value
    string normalized_url
}}
```

```
FRIEND_REQUEST {{
    string id
    string fromName
    string fromHandle
    string fromLink
    datetime createdAt
}}
```

```
FRIENDSHIP {{
    string id
    string name
    string handle
    string link
}}
```

```
NOTIFICATION {{
    string id
    string type
    string title
    string actorHandle
    datetime createdAt
}}
```

```
TRIBE {{
    string tribeId
    string name
    string ownerId
    string ownerDisplayName
    string ownerHandle
    string themeColor
    datetime createdAt
    datetime updatedAt
}}
```

```
TRIBE_MEMBER {{
    string tribeId
    string userId
}}
```

```
    string displayName
    string handle
    string link
}}

TRIBE_INVITE {{
    string id
    string tribeId
    string tribeName
    string ownerId
    string ownerDisplayName
    string ownerHandle
    datetime createdAt
}}

TRIBE_JOIN_REQUEST {{
    string id
    string tribeId
    string tribeName
    string requesterId
    string requesterDisplayName
    string requesterHandle
    datetime createdAt
}}

TRIBE_MESSAGE {{
    string id
    string senderId
    string senderDisplayName
    string senderHandle
    string text
    datetime createdAt
}}

GAME_SCORE {{
    string userId
    string game
    integer highScore
    datetime updatedAt
}}

MEDIA_ASSET {{
    string ownerUserId
    string handle
    string field
    string mimeType
    string storagePath
    string fileName
}}

ACCOUNT_SETTINGS {{
    string userId
    string dashboardTheme
    string cursorMode
```

```

    string cursorColor
    boolean muteOutsideBio
    boolean sidebarCollapsed
  }}

```

D. Sequence Diagrams

1. User Sign Up and Login

```

mermaid
sequenceDiagram
    actor User
    participant Browser
    participant API as Node API
    participant Store as Local JSON or Supabase

    User->>Browser: Enter email and password
    Browser->>API: POST /api/signup
    API->>API: Validate email and password length
    API->>API: Hash password with crypto.scrypt and salt
    API->>Store: Create user record
    API->>Store: Create session token
    API-->>Browser: Return token and email
    Browser->>Browser: Store token in localStorage
    Browser->>API: GET /api/me with Authorization header
    API-->>Browser: Return user account metadata

```

2. Public Profile Publishing and Viewing

```

mermaid
sequenceDiagram
    actor Owner
    actor Visitor
    participant Browser
    participant API as Node API
    participant Store as Profile storage

    Owner->>Browser: Edit profile fields and click Publish profile
    Browser->>API: PUT /api/profiles/:handle with session token
    API->>Store: Check ownership or create owned profile
    API->>Store: Save profile JSON and profile URL fields
    API-->>Browser: Return /u/:handle and full URL
    Visitor->>Browser: Open /u/:handle
    Browser->>API: GET /api/profiles/:handle?view=1
    API->>Store: Increment profile view count
    API-->>Browser: Return public profile payload without owner/private fields

```

3. Upload Profile, Background, or Music Media

```

mermaid
sequenceDiagram
    actor Owner

```

```

participant Browser
participant API as Node API
participant Storage as Local JSON or Supabase Storage

Owner->>Browser: Select avatar, background, or music file
Browser->>Browser: Convert file to data URL for preview
Owner->>Browser: Publish profile
Browser->>API: PUT /api/profiles/:handle with media data
alt Supabase configured
  API->>Storage: Upload media to profile-media bucket
  API->>API: Replace data URL with storage path
else Local mode
  API->>Storage: Persist data URL in profile JSON
end
API-->>Browser: Return saved profile link
Browser->>API: GET /api/profiles/:handle/background or music
API-->>Browser: Stream media response

```

4. Send and Accept Friend Request

```

mermaid
sequenceDiagram
    actor Sender
    actor Receiver
    participant Browser
    participant API as Node API
    participant Store as Profile storage

    Sender->>Browser: Enter friend's handle or profile link
    Browser->>API: POST /api/friend-requests
    API->>Store: Validate sender profile and target profile
    API->>Store: Append request to target profile
    API->>Store: Append sent request to sender profile
    API-->>Browser: Return sent status
    Receiver->>Browser: Opens notifications or friend requests
    Browser->>API: POST /api/friend-requests/:id/accept
    API->>Store: Add each user to the other's friends list
    API->>Store: Remove request and sent request
    API-->>Browser: Return updated friends and requests

```

5. Create Tribe and Invite Friends

```

mermaid
sequenceDiagram
    actor Owner
    actor Friend
    participant Browser
    participant API as Node API
    participant Store as Profile storage

    Owner->>Browser: Create tribe with name, color, and invited friends
    Browser->>API: POST /api/tribes
    API->>Store: Create tribe under owner profile
    API->>Store: Add owner to memberIds

```

```
API->>Store: Add tribe invite records to invited friend profiles
API-->>Browser: Return updated tribe state
Friend->>Browser: Sees invite notification with Yes/No
Browser->>API: POST /api/tribe-invites/:id/accept
API->>Store: Add friend userId to tribe memberIds
API->>Store: Remove pending invite
API-->>Browser: Return updated tribe state
```

6. Request to Join Tribe and Owner Approval

```
mermaid
sequenceDiagram
    actor Requester
    actor Owner
    participant Browser
    participant API as Node API
    participant Store as Profile storage

    Requester->>Browser: Search tribe and click Join
    Browser->>API: POST /api/tribes/:tribeId/join
    API->>Store: Add requester to pendingJoinIds
    API->>Store: Add join request to owner profile
    API-->>Browser: Return requested status
    Owner->>Browser: Opens notifications or friend requests tab
    Browser->>API: POST /api/tribe-join-requests/:id/accept
    API->>Store: Add requester userId to tribe memberIds
    API->>Store: Remove pending join request
    API-->>Browser: Return updated tribe state
```

7. Open Tribe Chat and Send Message

```
mermaid
sequenceDiagram
    actor Member
    participant Browser
    participant API as Node API
    participant Store as Profile storage

    Member->>Browser: Open Your Tribe Chats and select tribe
    Browser->>API: GET /api/tribes/:tribeId/messages
    API->>Store: Confirm requester is owner or member
    API-->>Browser: Return messages for that tribe only
    Member->>Browser: Type message and send
    Browser->>API: POST /api/tribes/:tribeId/messages
    API->>Store: Sanitize text and append message
    API->>Store: Keep latest 300 messages
    API-->>Browser: Return updated message list
```

8. Save Game Score

```
mermaid
sequenceDiagram
    actor User
    participant Browser
```

participant API as Node API
participant Store as User storage

User->>Browser: Play Snake
Browser->>Browser: Calculate score and local best
Browser->>API: PUT /api/games/snake-score
API->>Store: Save max score for authenticated user
API->>Browser: Return highScore
Browser->>Browser: Show new best score

E. Data Flow Summary

Authentication/session flow:

- The browser submits credentials to 'POST /api/signup' or 'POST /api/login'.
- The server validates inputs, hashes or verifies the password, creates a random session token, and returns it.
- The browser stores the token in localStorage and sends it in the 'Authorization: Bearer' header.
- The server resolves the token through local JSON or Supabase sessions.

Profile update flow:

- The profile editor collects display name, handle, bio, location, social links, theme, cursor, sparkle, media, and toggle settings.
- The browser sends the profile payload to 'PUT /api/profiles/:handle'.
- The server verifies the session and profile ownership before saving.
- Public profile links are saved back onto the user record when possible.

Media upload flow:

- The frontend previews selected files as data URLs.
- In Supabase mode, the server uploads media to 'profile-media' and stores object paths.
- In local JSON mode, media remains embedded in profile JSON.
- Public profile media is served through '/api/profiles/:handle/avatar', '/background', and '/music'.

Friends/notifications flow:

- Friend requests are stored on the target profile and sent request state is stored on the sender profile.
- Notifications are rendered from friend requests, tribe invites, and tribe join requests.
- The frontend refreshes friend and notification state periodically.

Tribes flow:

- Tribes are owned by the creator and stored inside the owner's profile JSON.
- Tribe summaries are built by scanning all profiles and normalizing their tribe arrays.
- Owner-only actions are enforced by comparing the authenticated user ID with 'tribe.ownerId'.

Tribe chat flow:

- Tribe messages are scoped by 'tribeId'.
- Messages are stored inside the tribe object and capped at the latest 300 messages.
- Access is limited to the owner or member IDs.

Games score flow:

- Snake has server-backed high score storage.
- Click Rush and Crossy Road use localStorage best scores in the frontend.
- Wordle validates guesses against 'words-5.txt' with a fallback word list.

F. Security & Permissions

Confirmed controls:

- Passwords are hashed with Node 'crypto.scrypt' and a random salt.
- Session tokens are generated with 'crypto.randomBytes'.
- Private dashboard APIs require a valid bearer token.
- Profile publishing requires authentication.
- Existing profile ownership is enforced by 'ownerUserId'.
- Public profile reads return a public payload and remove owner/private fields.
- Public profile view increments happen only when '?view=1' is used.
- Tribe edit, delete, and member removal require tribe ownership.
- Tribe chat access requires the requester to be the owner or a member.
- The owner cannot remove themselves from their tribe through the member removal endpoint.
- Supabase service role key is used only on the server.

Assumptions and limitations:

- Session expiration is not visible in the current implementation.
- Password reset and email verification are not implemented.
- Uploaded media validation is based on accepted frontend file types and server MIME handling; deeper scanning is not implemented.
- Rate limiting is not implemented.
- Realtime authorization is not applicable because chat is currently request/response, not WebSocket-based.

G. Scalability Recommendations

- Move production persistence fully to Supabase instead of local JSON.
- Normalize high-volume entities into first-class tables: friendships, friend requests, tribes, tribe members, tribe messages, notifications, and media assets.
- Add database indexes for profile handle, owner user ID, tribe name search, tribe membership, and message timestamps.
- Add Supabase Row Level Security or equivalent API authorization policies if direct client access is ever introduced.
- Add session expiration, refresh, logout invalidation, and device/session management.
- Add rate limiting for auth, profile publishing, media uploads, friend requests, tribe joins, and chat messages.
- Add file size limits, content-type verification, media compression, and lifecycle cleanup for replaced media.
- Move tribe chat to Supabase Realtime or WebSockets when live chat behavior is required.
- Add audit logs for owner actions such as deleting tribes, removing members, and changing tribe settings.

- Add moderation workflows for public profiles, tribe names, chat messages, and uploaded media.
- Add structured API route modules as the backend grows beyond the current single-file server.

Product Design Document

Generated: 2026-05-24

Source inspection scope: 'index.html', 'script.js', 'styles.css', 'server.js', 'package.json', and 'supabase-schema.sql'.

1. Executive Summary

fun.lol is a customizable public profile platform where users can create a public bio page, add visual and audio media, connect with friends, join tribes, chat with tribe members, and play mini-games from a private dashboard. The current product blends a personal identity page with lightweight social and entertainment features.

2. Product Vision

The long-term vision is for fun.lol to become a highly personalized social identity hub. Users should be able to express themselves through media-rich profiles, share one public link, keep up with friends, create small communities, and enjoy interactive experiences without leaving the platform.

3. Target Users

- Creators who want a personalized profile link.
- Gamers who enjoy expressive profiles and mini-games.
- Social users who want a lightweight friend graph.
- Friend groups who want private-ish spaces around shared identity.
- Community or tribe members who want a shared group and chat.
- Public visitors who open a user's profile link.

4. Problem Statement

Many profile-link tools are static, generic, and disconnected from the user's actual social experience. fun.lol solves this by combining profile customization, media, friend requests, tribe communities, tribe chats, and mini-games in one lightweight web platform.

5. Product Goals

- Enable personal profile customization.
- Support public profile sharing by handle.
- Make social discovery and friend connection simple.

- Provide tribe/community engagement.
- Add mini-game entertainment inside the dashboard.
- Preserve a strong dark, animated, profile-first brand feel.

6. Non-Goals

The current version does not include:

- A full social media feed.
- Payment or subscription systems.
- Advanced moderation dashboard.
- Native mobile app.
- Enterprise admin portal.
- End-to-end encrypted messaging.
- Realtime chat infrastructure.
- Password reset or email verification.

7. Key Features

Account & Authentication

- Email and password sign up.
- Login with saved browser session.
- Auto-login for returning users.
- Server-side password hashing.
- Settings page with account and public profile details.

Dashboard

- Private logged-in home screen.
- Sidebar navigation for Bio, Games, Tribes, and Settings.
- Collapsible sidebar.
- Friends and notifications widgets.
- Dashboard theme selector.
- Pointer style and color controls.
- Mute Outside Bio control.

Bio/Profile Builder

- Editable display name, handle, bio, and location.
- Public profile publishing.
- Visitor preview.
- Public link generation.

- Profile view counter.
- Profile card with avatar, UID, featured section, location, views, and social buttons.

Profile Customization

- Avatar upload.
- Background image or video upload.
- Background music upload.
- Remove background and music files.
- Theme selection.
- Animated background, compact links, and dark video overlay toggles.
- Dot cursor, cursor color, and sparkle effects.

Music & Entry Experience

- Public profile entry screen before the profile opens.
- Music starts after user interaction.
- Music controls on the owner's bio editor.
- Public visitors do not see the full music information box.
- Public video loop transition with audio fade behavior.

Social Links

- Discord, Instagram, TikTok, YouTube, X, and GitHub social fields.
- Profile social output uses icons rather than plain text labels.

Friends & Notifications

- Add friends by name, handle, or profile link.
- Send and accept friend requests.
- Remove friends after confirmation.
- View sent friend requests.
- Notifications widget and friend request tab.
- Periodic refresh of friend and notification state.

Tribes

- Add Friends, Your Friends, Your Tribes, Join a Tribe, Your Tribe Chats, and Friend Requests tabs.
- Create tribes with a name and color.
- Invite friends to tribes.
- Accept or decline tribe invites.
- Search for tribes.
- Request to join tribes.
- Owner approval for join requests.
- Owner-only rename, color change, delete, and member removal.

Tribe Chats

- Grid of chats for tribes the user can access.
- Open a selected tribe chat.
- Send messages scoped to that tribe.
- Exit chat back to the grid.
- Non-members are blocked by the backend.

Games

- Games dashboard with card grid and expanded game views.
- Snake with server-backed high score.
- Click Rush with local best score.
- Wordle with English dictionary validation from 'words-5.txt'.
- Crossy Road-style game with increasing difficulty and obstacles.
- Memory Match and Orbit Dodge placeholders.

Backend/Data

- Node.js backend using built-in 'http'.
- Local JSON fallback storage.
- Optional Supabase database support.
- Optional Supabase Storage media bucket.
- API routes for auth, profiles, friends, tribes, tribe chats, media, games, sitemap, and robots.

Responsive UX

- Mobile layout adjustments.
- Custom cursor disabled on coarse pointer/mobile devices.
- Responsive dashboard/sidebar behavior.
- Glassmorphism dark UI.
- Animated star/void-style background.
- Hover movement effects.
- Loading, welcome, and entry transitions.

8. Functional Requirements

Authentication

1. The system shall allow a user to sign up with email and password.
1. The system shall reject invalid emails and passwords shorter than the configured minimum.
1. The system shall hash passwords before storing them.
1. The system shall allow a user to log in with a valid email and password.
1. The system shall return a session token after successful signup or login.

1. The system shall load account metadata through 'GET /api/me'.

Dashboard

1. The system shall show the authenticated dashboard only after login.
1. The system shall provide Bio, Games, Tribes, and Settings navigation.
1. The system shall allow the sidebar to collapse and expand.
1. The system shall persist dashboard theme, cursor mode, cursor color, mute setting, and sidebar state in localStorage.
1. The system shall show friends and notifications widgets on the home screen.

Profile Builder

1. The system shall allow users to edit name, handle, bio, and location.
1. The system shall normalize handles before publishing.
1. The system shall prevent authenticated users from editing profiles owned by another user.
1. The system shall generate '/u/:handle' public profile paths.
1. The system shall increment public views when a profile is opened with '?view=1'.

Media

1. The system shall allow avatar, background, and music upload from the browser.
1. The system shall preview selected media before publish.
1. The system shall store media in Supabase Storage when Supabase is configured.
1. The system shall serve media through profile media API endpoints.
1. The system shall allow background and music files to be removed.

Friends and Notifications

1. The system shall send friend requests to valid published profile handles.
1. The system shall prevent self friend requests.
1. The system shall prevent duplicate friendships from being added.
1. The system shall allow recipients to accept friend requests.
1. The system shall remove accepted requests from pending lists.
1. The system shall allow friends to be removed after confirmation.
1. The system shall refresh friend and notification state periodically.

Tribes

1. The system shall allow authenticated profile owners to create tribes.
1. The system shall allow tribe owners to invite existing friends.
1. The system shall show tribe invites as notifications.
1. The system shall allow invited users to accept or decline.
1. The system shall allow users to search tribes by name.
1. The system shall allow users to request to join a tribe.
1. The system shall allow tribe owners to accept or decline join requests.

1. The system shall allow owners to rename tribes and change tribe color.
1. The system shall allow owners to delete tribes.
1. The system shall allow owners to remove members except themselves.

Tribe Chats

1. The system shall show chat cards only for tribes the user owns or belongs to.
1. The system shall load messages for the selected tribe only.
1. The system shall reject chat access for non-members.
1. The system shall reject empty chat messages.
1. The system shall append sent messages to the selected tribe chat.
1. The system shall keep chat state when exiting back to the chat grid.

Games

1. The system shall allow users to open games from a card grid.
1. The system shall expand the selected game and provide a return action.
1. The system shall show how-to-play content for each playable game.
1. The system shall save Snake high score through the backend.
1. The system shall save Click Rush and Crossy Road best scores locally.
1. The system shall validate Wordle guesses against a dictionary.

9. Non-Functional Requirements

Security:

- Passwords must be hashed before storage.
- Private APIs must require valid sessions.
- Owner-only actions must be checked on the server.

Performance:

- Public profiles should minimize payload size by avoiding inline media in public view mode when storage paths exist.
- Background videos should load without blocking core profile text.

Scalability:

- Production should use Supabase or another managed database instead of local JSON.
- High-volume data such as chat messages should move to dedicated tables.

Availability:

- The app can run locally with JSON storage and in hosted mode with Supabase.
- Recommendation: add backups and monitoring for production.

Responsiveness:

- Dashboard and profile layouts should adapt to mobile and desktop.
- Custom cursor effects should be disabled on coarse pointer devices.

Accessibility:

- Interactive controls should preserve labels or accessible names.
- Recommendation: add keyboard and screen reader audits for games and dialogs.

Maintainability:

- Recommendation: split the growing 'server.js' and 'script.js' into modules as the product grows.

Browser compatibility:

- The app should work in modern Chromium, Edge, Safari, and Firefox browsers.
- Recommendation: test media autoplay behavior across browsers because music requires user interaction.

Data privacy:

- Public profile payloads should not include password hashes, session tokens, or private owner fields.
- Recommendation: avoid exposing Supabase service keys to the client.

Error handling:

- API errors should return clear messages.
- UI should show toast or status messages for failed actions.

10. User Roles & Permissions

Role: Public visitor; Description: Not logged in, opens a public profile; Key permissions: View public profile and media entry experience.

Role: Logged-in user; Description: Authenticated account holder; Key permissions: Access dashboard, publish own profile, use games, manage friends and tribes.

Role: Friend; Description: User connected through accepted friend request; Key permissions: Appears in friends list and can be invited to tribes.

Role: Tribe member; Description: User included in a tribe member list; Key permissions: View tribe details and access tribe chat.

Role: Tribe owner; Description: Creator/owner of a tribe; Key permissions: Rename, recolor, delete tribe, approve joins, remove members, access chat.

Role: Invited user; Description: User who received a tribe invite; Key permissions: Accept or decline the invite.

Role: Join requester; Description: User who requested tribe membership; Key permissions: Wait for owner approval; cannot access chat until accepted.

11. UX/UI Principles

- Dark glassmorphism visual language.
- Animated star/void background as a core brand element.
- Compact dashboard layout with clear navigation.
- Profile-first personalization with live preview behavior.
- Smooth loading, welcome, and entry transitions.
- Game-like interaction patterns for cards, hover motion, and cursor effects.
- Mobile-friendly behavior with custom cursor effects disabled where inappropriate.

12. Data Model

Object: User; Key fields: id, email, passwordHash/password_hash, profileHandle, profilePath, profileUrl, snakeHighScore; Current persistence: 'users.json' or 'app_users'.

Object: Profile; Key fields: handle, ownerId, name, bio, location, theme, views, updatedAt; Current persistence: 'profiles.json' or 'app_profiles.data'.

Object: AccountSettings; Key fields: dashboardTheme, cursorMode, cursorColor, muteOutsideBio, sidebarCollapsed; Current persistence: Browser localStorage.

Object: SocialLink; Key fields: platform, value, normalizedUrl; Current persistence: Profile JSON.

Object: FriendRequest; Key fields: id, fromName, fromHandle, fromLink, createdAt; Current persistence: Target profile JSON.

Object: Friendship; Key fields: id, name, handle, link; Current persistence: Profile JSON.

Object: Notification; Key fields: derived type, actor, target, createdAt; Current persistence: Derived from friend requests, tribe invites, and join requests.

Object: Tribe; Key fields: tribeId, name, ownerId, ownerDisplayName, ownerHandle, memberIds, pendingInviteIds, pendingJoinIds, themeColor, messages, createdAt, updatedAt; Current persistence: Owner profile JSON.

Object: TribeMember; Key fields: userId, displayName, handle, link; Current persistence: Derived from memberIds and profile data.

Object: TribeInvite; Key fields: id, tribeId, tribeName, ownerId, ownerDisplayName, ownerHandle, createdAt; Current persistence: Invited user's profile JSON.

Object: TribeJoinRequest; Key fields: id, tribeId, tribeName, requesterId, requesterDisplayName, requesterHandle, createdAt; Current persistence: Owner profile JSON.

Object: TribeMessage; Key fields: id, senderId, senderDisplayName, senderHandle, text, createdAt; Current persistence: Tribe object in owner profile JSON.

Object: GameScore; Key fields: userId, game, highScore; Current persistence: Snake in user storage; other best scores in localStorage.

Object: MediaAsset; Key fields: avatar/background/music data or path, name, type; Current persistence: Profile JSON plus optional Supabase Storage object.

13. Key User Flows

- Sign up: User enters email/password, server creates account and session, dashboard loads.
- Login: User enters credentials, server verifies password, session token is stored.
- Edit bio profile: User changes profile fields, preview updates, user publishes.
- Upload media: User selects avatar/background/music, preview updates, profile publish stores media.
- Publish public profile: Server saves profile and returns '/u/:handle'.
- Visitor opens profile: Visitor enters profile page, view count increments, public profile loads.
- Add friend: User enters a handle/link, server sends request to target profile.
- Accept friend request: Recipient accepts, both friend lists update.
- Create tribe: Owner creates tribe, optionally invites friends.
- Invite friend to tribe: Friend receives tribe invite notification and accepts/declines.
- Join tribe: User searches tribe and sends join request.
- Approve join request: Owner accepts requester into member list.
- Open tribe chat: Member selects chat card and loads scoped messages.
- Send tribe message: Member sends non-empty text, message is appended to tribe.
- Play game: User opens game, plays, game-over state shows score.
- Save score: Snake score saves through backend; selected other games use localStorage.
- Change dashboard/theme/cursor settings: User changes controls and settings persist locally.

14. Edge Cases

- Duplicate handle: Server rejects editing a profile owned by another account.
- Invalid profile link: Friend request route returns an error for invalid target handles.
- Failed media upload: Server returns an error and profile publish should show a failure message.

- Removed background/music file: UI clears local state and publish persists removal.
- Empty bio fields: Defaults are applied for name, handle, bio, and location.
- Friend already added: Existing friendship prevents duplicate request behavior.
- Duplicate friend request: Existing pending request is reused.
- Tribe already exists: Current implementation does not enforce unique tribe names. Recommendation: add duplicate-name handling per owner.
- User already in tribe: Join route returns joined status.
- User already invited: Duplicate invite records are avoided.
- Non-member accessing tribe chat: Server returns a forbidden error.
- Owner removing themselves: Server rejects removing the owner.
- Tribe deleted while chat is open: Active chat is cleared when tribe state refresh removes access.
- Empty chat message: Server rejects empty sanitized text.
- Game score save failure: Snake falls back to local best-score behavior in the frontend.
- Local JSON storage unavailable: Server file read/write would fail. Recommendation: add operational monitoring and backups.
- Supabase unavailable: API requests will fail in Supabase mode. Recommendation: add retry and user-friendly status handling.

15. Acceptance Criteria

Authentication:

- A new user can sign up and reaches the dashboard.
- Existing users can log in without recreating accounts.
- Invalid credentials produce a clear error.

Dashboard:

- Sidebar navigation changes active sections.
- Sidebar collapse state persists.
- Cursor, theme, and mute controls do not overlap and remain clickable.

Profile:

- A user can publish a profile and receive a public link.
- Visitors can open '/u/:handle' without edit controls.
- View count increases when a public profile is viewed.

Media:

- Avatar, background, and music can be selected, previewed, saved, and loaded again.
- Remove buttons clear background and music.

Friends:

- A user can send a friend request by handle or profile link.
- Recipient can accept the request.
- Friends appear without requiring a full page refresh after periodic refresh.

- Removing a friend asks for confirmation.

Tribes:

- A user can create a tribe and invite friends.
- Invited users receive Yes/No notifications.
- Users can search and request to join tribes.
- Owners can approve join requests.
- Owners can rename, recolor, delete, and remove members.
- Non-owners do not see or use owner controls.

Tribe Chats:

- Members see chat cards for their tribes.
- Non-members cannot open a tribe chat.
- Messages are scoped to the active tribe.
- Exit Chat returns to the chat grid.

Games:

- Playable games open from the grid.
- Snake, Click Rush, Wordle, and Crossy Road show game-over states.
- Wordle rejects guesses not found in the dictionary.
- Snake high score is saved for the authenticated user.

16. Analytics & Success Metrics

- Signups.
- Logins and returning users.
- Published profiles.
- Public profile views.
- Media uploads.
- Profile publish success rate.
- Friend requests sent and accepted.
- Active friendships.
- Tribes created.
- Tribe invites accepted/declined.
- Tribe join requests accepted/declined.
- Tribe chat messages sent.
- Games opened.
- Games completed.
- Snake high score saves.
- Theme/cursor customization usage.

17. Risks & Mitigations

Risk: Local JSON storage may not scale; Type: Technical; Mitigation: Move production data to Supabase tables and add backups.

Risk: Chat stored inside owner profile JSON can become large; Type: Technical; Mitigation: Move messages to a dedicated messages table or realtime service.

Risk: No session expiration visible; Type: Security; Mitigation: Add token expiry, refresh, and logout invalidation.

Risk: Public media may be large; Type: Performance; Mitigation: Add upload limits, compression, and CDN-backed storage.

Risk: User-generated content can be abusive; Type: Product/Safety; Mitigation: Add reporting, moderation, block lists, and content rules.

Risk: Music/video autoplay varies by browser; Type: UX; Mitigation: Keep entry gate and test browser-specific media behavior.

Risk: Large 'script.js' and 'server.js' reduce maintainability; Type: Engineering; Mitigation: Split by feature modules and add automated tests.

18. Future Enhancements

- Realtime tribe chat.
- Push notifications.
- Image/audio attachments in chat.
- Tribe privacy controls.
- Moderation tools and report flows.
- Profile templates.
- More games.
- Public tribe discovery.
- Supabase-first production database with normalized social tables.
- Mobile app.
- Premium customization features.
- Password reset and email verification.
- Admin analytics dashboard.

Product Journey Map

Generated: 2026-05-24

Source inspection scope: 'index.html', 'script.js', 'styles.css', 'server.js', 'package.json', and 'supabase-schema.sql'.

Journey 1: New User Onboarding

Stage: Discover fun.lol; User Goal: Understand what the platform does; User Action: Opens the site; System Response: Shows auth screen with profile creation message; User Emotion: Curious; Pain Point: Value proposition may depend on seeing examples; Opportunity / Improvement: Add optional public demo profiles or sample preview; Success Metric: Landing-to-signup conversion

Stage: Sign up; User Goal: Create an account; User Action: Enters email and password; System Response: Server creates user and session; User Emotion: Hopeful; Pain Point: Password rules are basic; Opportunity / Improvement: Add inline password guidance; Success Metric: Signup completion rate

Stage: Login; User Goal: Access account; User Action: Enters credentials; System Response: Server verifies password and returns token; User Emotion: Relieved; Pain Point: No password reset in current version; Opportunity / Improvement: Add reset flow and email verification; Success Metric: Login success rate

Stage: Land on dashboard; User Goal: Know where to start; User Action: Arrives at dashboard home; System Response: Shows welcome, friends widget, notifications widget, controls; User Emotion: Oriented; Pain Point: New users may not know Bio is first step; Opportunity / Improvement: Add first-run checklist; Success Metric: Bio editor open rate

Stage: Explore sidebar; User Goal: Navigate features; User Action: Clicks Bio, Games, Tribes, Settings; System Response: Dashboard switches panels; User Emotion: Interested; Pain Point: Mobile navigation can feel dense; Opportunity / Improvement: Add onboarding hints and responsive polish; Success Metric: Sidebar engagement

Stage: Start editing bio; User Goal: Build identity; User Action: Opens Bio editor; System Response: Shows profile card and editing forms; User Emotion: Creative; Pain Point: Long editor can feel complex; Opportunity / Improvement: Group fields and offer templates; Success Metric: First profile edit rate

Stage: Publish public profile; User Goal: Share identity; User Action: Clicks Publish profile; System Response: Server saves profile and returns public link; User Emotion: Accomplished; Pain Point: Handle conflicts can block publish; Opportunity / Improvement: Add handle availability check; Success Metric: Published profiles

Journey 2: Profile Customization

Stage: Edit display name/handle; User Goal: Personalize identity; User Action: Types name and handle; System Response: Preview updates and publish payload changes; User Emotion: In control; Pain Point: Duplicate handles are found on publish; Opportunity / Improvement: Add live handle validation; Success Metric: Successful handle saves

Stage: Add bio/location; User Goal: Add context; User Action: Enters bio and location; System Response: Profile text updates; User Emotion: Expressive; Pain Point: Empty states use defaults; Opportunity / Improvement: Add optional prompts; Success Metric: Bio completion rate

Stage: Upload avatar; User Goal: Show personality; User Action: Selects image; System Response: Avatar preview updates; User Emotion: Excited; Pain Point: Large image may slow save; Opportunity / Improvement: Add image compression; Success Metric: Avatar upload count

Stage: Upload background image/video; User Goal: Make page immersive; User Action: Selects media file; System Response: Background preview updates; User Emotion: Impressed; Pain Point: Large videos can load slowly; Opportunity / Improvement: Add file size guidance and compression; Success Metric: Background upload count

Stage: Upload music; User Goal: Add sound; User Action: Selects audio file; System Response: Music player becomes available; User Emotion: Playful; Pain Point: Browser requires interaction before playback; Opportunity / Improvement: Keep entry gate and explain only when needed; Success Metric: Music upload count

Stage: Choose theme/cursor/sparkle effects; User Goal: Tune style; User Action: Selects theme, cursor, color, sparkle; System Response: UI updates immediately; User Emotion: Creative; Pain Point: Many controls can crowd UI; Opportunity / Improvement: Keep separate control groups; Success Metric: Customization usage

Stage: Preview profile; User Goal: See visitor view; User Action: Clicks visitor preview; System Response: Edit controls hide for preview; User Emotion: Confident; Pain Point: Preview can differ from public media behavior; Opportunity / Improvement: Add public-preview checklist; Success Metric: Preview usage

Stage: Share public profile link; User Goal: Bring visitors; User Action: Copies or opens '/u/:handle'; System Response: Public page loads and increments views; User Emotion: Proud; Pain Point: Sharing depends on remembering link; Opportunity / Improvement: Add share buttons; Success Metric: Profile views

Journey 3: Friends & Notifications

Stage: Search for friend; User Goal: Find someone; User Action: Enters friend handle or profile link; System Response: Form validates target after submit; User Emotion: Focused; Pain Point: No full people search yet; Opportunity / Improvement: Add user search and suggestions; Success Metric: Friend search attempts

Stage: Send request; User Goal: Connect; User Action: Submits friend request; System Response: Target profile receives request; sender sees sent state; User Emotion: Hopeful; Pain Point: Invalid handle creates failure; Opportunity / Improvement: Add autocomplete; Success Metric: Requests sent

Stage: Friend receives notification; User Goal: Notice incoming request; User Action: Opens dashboard; System Response: Notification widget and request tab show request; User Emotion: Aware; Pain Point: Notifications are pull-based; Opportunity / Improvement: Add realtime or push notifications; Success Metric: Request open rate

Stage: Friend accepts; User Goal: Confirm connection; User Action: Clicks accept; System Response: Both friend lists update; User Emotion: Satisfied; Pain Point: Decline flow is not as prominent as accept; Opportunity / Improvement: Add clearer action states; Success Metric: Acceptance rate

Stage: Friends list updates; User Goal: See current network; User Action: Waits or navigates; System Response: Frontend refreshes about every 10 seconds; User Emotion: Reassured; Pain Point: Delay may feel slow; Opportunity / Improvement: Use realtime subscriptions later; Success Metric: Refresh success rate

Stage: Remove friend if needed; User Goal: Control network; User Action: Clicks remove; System Response: Confirmation dialog appears; friendship removed if confirmed; User Emotion: Safe; Pain Point: Mistakes are possible; Opportunity / Improvement: Keep confirmation and add undo later; Success Metric: Friend removals

Journey 4: Tribes & Tribe Chats

Stage: Open Tribes section; User Goal: Manage community; User Action: Clicks Tribes; System Response: Shows top tabs for friends, tribes, join, chats, requests; User Emotion: Curious; Pain Point: Many tabs can feel dense; Opportunity / Improvement: Add clearer grouping or icons; Success Metric: Tribes tab visits

Stage: View friends/tribes tabs; User Goal: Understand options; User Action: Switches tabs; System Response: Shows relevant list panels; User Emotion: Oriented; Pain Point: Empty states need guidance; Opportunity / Improvement: Add next-action empty states; Success Metric: Tab engagement

Stage: Create tribe; User Goal: Start a group; User Action: Enters name, color, invited friends; System Response: Server creates tribe under owner profile; User Emotion: Proud; Pain Point: Tribe names are not globally unique; Opportunity / Improvement: Add owner-level or global uniqueness rules; Success Metric: Tribes created

Stage: Invite friends; User Goal: Grow tribe; User Action: Selects friends during creation; System Response: Invited users receive tribe invite notification; User Emotion: Social; Pain Point: Invites require existing friendship; Opportunity / Improvement: Add invite by profile handle later; Success Metric: Invites sent

Stage: Friend accepts tribe invite; User Goal: Join group; User Action: Clicks Yes; System Response: User is added to memberIds; User Emotion: Included; Pain Point: Declined state is simply removed; Opportunity / Improvement: Add owner-facing invite status; Success Metric: Invite acceptance rate

Stage: Search/join tribe; User Goal: Discover groups; User Action: Searches and clicks Join; System Response: Owner receives join request; User Emotion: Interested; Pain Point: No public tribe categories; Opportunity / Improvement: Add discovery filters; Success Metric: Join requests

Stage: Owner approves join request; User Goal: Manage access; User Action: Clicks accept; System Response: Requester becomes member; User Emotion: Responsible; Pain Point: Owner must check notifications manually; Opportunity / Improvement: Add notification badge improvements; Success Metric: Join approval rate

Stage: Open Your Tribe Chats; User Goal: Chat with group; User Action: Clicks chat tab; System Response: Shows accessible tribe chat cards; User Emotion: Connected; Pain Point: No realtime updates; Opportunity / Improvement: Add realtime chat; Success Metric: Chat opens

Stage: Select tribe chat grid card; User Goal: Focus conversation; User Action: Clicks tribe card; System Response: Loads only that tribe messages; User Emotion: Engaged; Pain Point: Messages are capped but not searchable; Opportunity / Improvement: Add search later; Success Metric: Messages loaded

Stage: Send message; User Goal: Participate; User Action: Enters text and sends; System Response: Message appends to active tribe chat; User Emotion: Social; Pain Point: Text-only messages; Opportunity / Improvement: Add attachments and reactions; Success Metric: Messages sent

Stage: Exit chat; User Goal: Return to overview; User Action: Clicks Exit Chat; System Response: Returns to chat grid without deleting messages; User Emotion: In control; Pain Point: State may become stale; Opportunity / Improvement: Refresh active chat periodically; Success Metric: Exit success

Journey 5: Games

Stage: Open Games dashboard; User Goal: Find entertainment; User Action: Clicks Games; System Response: Shows scrollable game grid; User Emotion: Playful; Pain Point: Coming-soon games may disappoint; Opportunity / Improvement: Clearly label playable vs coming soon; Success Metric: Games tab visits

Stage: Browse game cards; User Goal: Choose game; User Action: Scans cards; System Response: Cards show names and previews; User Emotion: Curious; Pain Point: Game previews could be more descriptive; Opportunity / Improvement: Add short metadata on hover; Success Metric: Game card clicks

Stage: Expand game; User Goal: Start focused play; User Action: Clicks game card; System Response: Selected game expands with controls; User Emotion: Excited; Pain Point: Large cards need responsive sizing; Opportunity / Improvement: Continue mobile tuning; Success Metric: Game opens

Stage: Read how-to-play; User Goal: Learn rules; User Action: Reads side help; System Response: Each game shows individual instructions; User Emotion: Prepared; Pain Point: Instructions may be missed; Opportunity / Improvement: Keep help close to game area; Success Metric: Help visibility

Stage: Play game; User Goal: Have fun; User Action: Uses mouse/keyboard; System Response: Game updates score and state; User Emotion: Focused; Pain Point: Keyboard controls vary by device; Opportunity / Improvement: Add touch controls for mobile; Success Metric: Game starts

Stage: Save/view score; User Goal: Track progress; User Action: Finishes game; System Response: Snake saves to backend; other games use local best; User Emotion: Competitive; Pain Point: Not all scores are server-backed; Opportunity / Improvement: Add unified game score model; Success Metric: Score saves

Stage: Return to dashboard; User Goal: Continue exploring; User Action: Clicks Back to games or sidebar; System Response: Returns to grid or dashboard; User Emotion:

Satisfied; Pain Point: None major; Opportunity / Improvement: Add recent games widget later; Success Metric: Repeat play rate

Mermaid User Journey Diagram

```
mermaid
journey
    title fun.lol User Journey
    section Onboarding
        Discover fun.lol: 3: Visitor
        Sign up: 4: New user
        Land on dashboard: 4: New user
        Publish public profile: 5: New user
    section Customization
        Edit profile identity: 5: User
        Upload media: 4: User
        Choose effects: 5: User
        Share link: 5: User
    section Friends
        Send friend request: 4: User
        Accept request: 5: Friend
        Friends list updates: 4: User
    section Tribes
        Create tribe: 5: Owner
        Invite friends: 4: Owner
        Join tribe: 4: Member
        Send tribe chat message: 5: Member
    section Games
        Browse games: 4: User
        Play game: 5: User
        View score: 4: User
```

Cross-Journey Recommendations

- Add first-run onboarding to guide new users toward publishing their first profile.
- Add live handle availability checks before publish.
- Add media size guidance before upload.
- Add realtime or near-realtime notifications for friend and tribe events.
- Add a unified server-backed score model for all games.
- Add moderation and reporting before scaling public discovery.